ROS 2 Nav2 SLAM Toolbox Gazebo

SignalSCOUT

Autonomous Spatial WiFi Signal Mapper — Technical Report

ROS 2 pipeline: SLAM mapping → Nav2 autonomous navigation → simulated RSSI sensing → RBF-interpolated coverage heatmaps

Repository: `ros2_RSSI_Heatmap`

Report generated for the completed implementation, including both bonus challenges

Table of Contents

1. Executive Summary

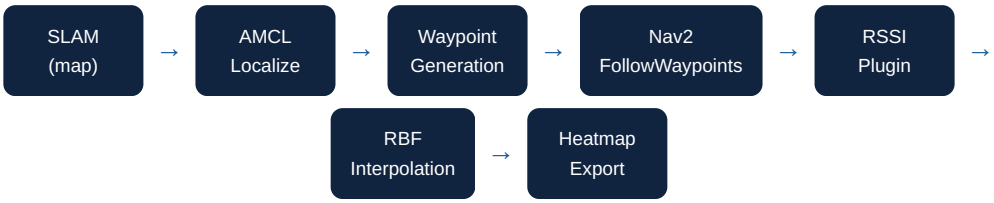
SignalSCOUT is a ROS 2 pipeline that turns a mobile robot into an autonomous WiFi site-survey instrument. A TurtleBot3 in Gazebo builds an occupancy-grid map with SLAM Toolbox, localizes against that map with Nav2's AMCL-based stack, autonomously visits a dense, collision-free grid of survey points via Nav2's `FollowWaypoints` action, samples a physically-modeled WiFi signal at each point through a custom Nav2 waypoint-task plugin, and finally interpolates the sparse per-waypoint samples into a continuous, colour-mapped coverage heatmap overlaid on the original map.

This report documents every piece of engineering work completed against the project's specification: the four required implementation tasks (waypoint generation, Nav2 goal dispatch, RSSI physical simulation, and heatmap visualization/export) plus **both** optional bonus challenges (multi-access-point coverage modelling, and nearest-neighbour dynamic route planning). For every component this report explains *what* was built, *why* it was built that way, the exact algorithm/formula involved, and the concrete code that implements it.

> **Scope completed:** all 4 core TODOs, Bonus Challenge 1 (multi-AP), and Bonus Challenge 2 (nearest-first navigation) are implemented across 5 ROS 2 packages (2 Python, 1 C++/pluginlib, 1 interface package, 1 bringup/launch package).

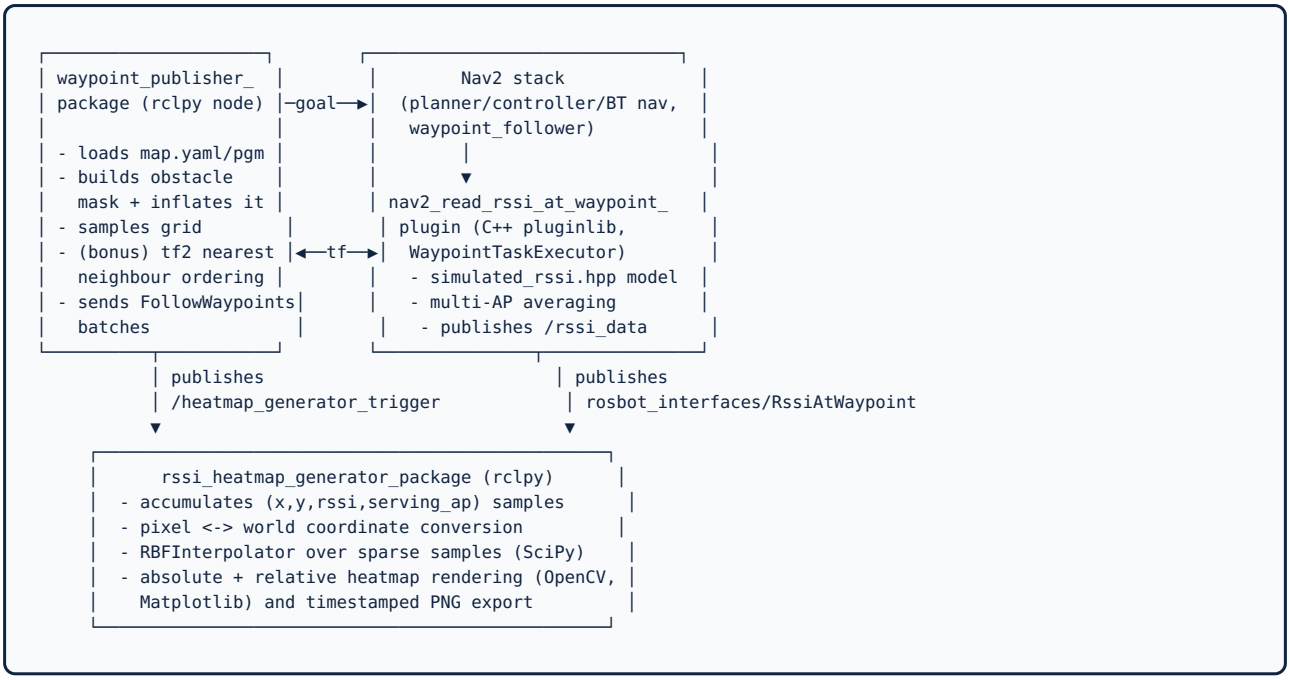
2. Problem Statement & Motivation

Manual WiFi site surveys are slow, labour-intensive, and hard to reproduce: an engineer walks the entire floor with a signal meter, logging readings at dozens or hundreds of points, then repeats the whole process whenever the network layout changes. SignalSCOUT automates this workflow end-to-end: a robot maps the space once via SLAM, autonomously plans and executes a survey route with Nav2, records a simulated RSSI value at every stop, and produces two heatmaps (absolute signal strength and relative-to-this-survey signal strength) that a network administrator could use to spot dead zones and evaluate access-point placement, without any human walking the floor.



3. System Architecture

The system is decomposed into five ROS 2 packages, each with a single clear responsibility. Nodes communicate exclusively through topics, an action interface, and one custom message type — there is no shared memory or tight coupling between the survey logic and the RSSI physics model, which is what allows the RSSI plugin to run *inside* Nav2's own waypoint_follower process while the heatmap generator runs as a fully independent node.



Supporting packages: `rosbot_interfaces` (the shared `RssiAtWaypoint.msg` definition) and `mappers_bringup` (all launch files and the Nav2 / SLAM Toolbox parameter YAMLs).

4. Package Inventory

Package	Type	Responsibility
`mappers_bringup`	ament_cmake (launch-only)	Launch files for Gazebo/SLAM/Nav2 bring-up and the survey pipeline; Nav2 & SLAM Toolbox parameter YAMLs.
`waypoint_publisher_package`	ament_python	Generates safe survey waypoints from the saved occupancy map and drives the robot through them via Nav2's `FollowWaypoints` action.
`nav2_read_rssi_at_waypoint_plugin`	ament_cmake (C++ pluginlib)	A `nav2_core::WaypointTaskExecutor` plugin invoked by Nav2's waypoint_follower each time a waypoint is reached; simulates and publishes RSSI.
`rosbot_interfaces`	ament_cmake (interfaces)	Defines the custom `RssiAtWaypoint` message shared between the plugin and the heatmap generator.
`rssi_heatmap_generator_package`	ament_python	Aggregates RSSI samples, interpolates them into continuous heatmaps, visualizes and saves the results.

5. Coordinate Frame Conventions

A recurring subtlety in this project is that **three different coordinate systems** have to be reconciled consistently: the ROS `map` frame (metres, origin and axis orientation defined by `map.yaml`), the occupancy-grid **pixel** frame (origin at the image's top-left corner, Y growing downward), and OpenCV's in-memory array indexing (`image[row][col]`, i.e. `image[y][x]`). Getting this wrong is the single most common source of "waypoints appear mirrored/shifted" bugs in this class of project, so it is documented explicitly here.

5.1 World → Pixel (used by both the waypoint publisher and the heatmap generator's RSSI callback)

```
px = (world_x − origin_x) / resolution     py = image_height − (world_y − origin_y) / resolution
```

The Y-axis is inverted because `map.yaml`'s `origin` describes the map-frame coordinate of the image's **bottom-left** pixel (standard ROS convention, matching how `map_server` saves PGMs), while image/array row indices increase downward from the top. Every pixel→world and world→pixel conversion in the codebase applies this same flip consistently, which is what keeps the waypoint publisher's generated grid, the RSSI plugin's world-frame poses, and the heatmap generator's pixel-space overlay all in registration with each other.

5.2 Pixel → World (used by the waypoint publisher when converting a sampled grid cell into a Nav2 goal)

```
world_x = origin_x + px × resolution     world_y = origin_y + (image_height − py) × resolution
```

This is the exact algebraic inverse of §5.1, applied when turning an accepted grid-sample pixel into the `PoseStamped` goal that gets sent to Nav2.

6. Waypoint Generation Engine Python

File: `waypoint_publisher_package/waypoint_publisher_package/node.py`, method `_generate_waypoints()`. This is the piece of the pipeline responsible for turning a raw occupancy-grid image into a safe, evenly-spaced set of navigation targets.

6.1 Step-by-step algorithm

1. **Grayscale conversion.** The BGR map image loaded via `cv2.imread` is converted to a single grayscale channel with `cv2.cvtColor(self.map, cv2.COLOR_BGR2GRAY)`, since the occupancy value is encoded purely in pixel intensity (free space ≈254, obstacle ≈0, unknown ≈205 for a standard `map_server`-saved PGM).
2. **Obstacle mask construction.** Any pixel whose intensity is below 250 is conservatively treated as *not free space* — this single threshold captures both hard obstacles (near-black) and unknown/unmapped regions (mid-gray ≈205) in one boolean mask, which is a deliberate simplification: the robot should not be sent into regions the map has no information about, exactly as it should not be sent into a wall.
3. **Safety-margin inflation.** The raw obstacle mask is dilated with `cv2.dilate(obstacle_mask, kernel)` using an **elliptical structuring element** whose size is derived from the `collision_range` ROS parameter (`kernel_size = 2*collision_range + 1`). This is a classic morphological-inflation technique: every obstacle pixel "grows" outward by `collision_range` pixels in all directions, so any candidate waypoint that survives this test is guaranteed to be at least that many pixels away from the nearest wall — a computationally cheap stand-in for a full costmap inflation layer.
4. **Uniform grid sampling.** Rather than evaluating every pixel (which would be wasteful and produce far more waypoints than needed), the algorithm strides through the image in steps of `density` pixels in both axes (`range(0, height, self.density)` / `range(0, width, self.density)`), producing a regular lattice of candidate points whose spacing is directly configurable via the `density` parameter.
5. **Accept/reject classification.** Each sampled pixel is looked up in the *inflated* obstacle mask: a value of 0 means free space outside the safety margin (accepted — colored green in the output visualization for QA), any non-zero value means too close to an obstacle or on one (rejected — colored red).
6. **Pixel → world conversion & storage.** Every accepted point is converted to map-frame coordinates using the formula in §5.2 and appended to `self.robot_frame_waypoint_array` as a `Waypoint(x, y)` namedtuple.

6.2 Implementation

```
gray = cv2.cvtColor(self.map, cv2.COLOR_BGR2GRAY)
height, width = gray.shape

# Free space is bright (254), obstacles are dark (0), unknown is mid-gray (205).
# Treat everything that is not clearly free space as occupied.
obstacle_mask = (gray < 250).astype(np.uint8)

kernel_size = 2 * self.collision_range + 1
kernel = cv2.getStructuringElement(cv2.MORPH_ELLIPSE, (kernel_size, kernel_size))
inflated_obstacle_mask = cv2.dilate(obstacle_mask, kernel)

for y in range(0, height, self.density):
    for x in range(0, width, self.density):
        if inflated_obstacle_mask[y, x] == 0:
            self.map[y, x] = (0, 255, 0)   # green: accepted
            world_x = self.origin.x + x * self.resolution
            world_y = self.origin.y + (height - y) * self.resolution
            self.robot_frame_waypoint_array.append(Waypoint(world_x, world_y))
        else:
            self.map[y, x] = (255, 0, 0)   # red: rejected (too close to obstacle)
```

> **Design note — colour channel ordering.** Although `cv2.imread` loads pixels in BGR order, this codebase's downstream visualization path (`show_map` / `plt.imshow`, and the heatmap generator's `add_waypoints` / `add_heatmap` helpers) treats the in-memory array as **RGB** from this point forward, only converting back to BGR immediately before `cv2.imwrite`. The accept/reject colours were therefore deliberately written

> as `(0,255,0)` and `(255,0,0)` — true green and true red under an RGB interpretation — rather than the BGR-correct tuples, so that the on-screen `matplotlib` display (which never applies a BGR→RGB conversion) renders the intended colours.

7. Nav2 Navigation Integration Python

File: `waypoint_publisher_package/waypoint_publisher_package/node.py`, method `_send_next_batch()`. Once a full list of waypoints exists, they must be converted into the message type Nav2's `FollowWaypoints` action actually expects and dispatched through an `rclpy` action client.

7.1 Batching

Waypoints are not sent all at once: they are chunked into `batch_size`-sized groups (a configurable launch parameter, default 20). This bounds the size of any single Nav2 goal, keeps feedback/logging granular, and — critically for Bonus 2 (§10) — creates a natural point at which the robot's live position can be re-queried before deciding the next set of waypoints to visit.

7.2 PoseStamped construction

```
msg = FollowWaypoints.Goal()

poses = []
for waypoint in batch:
    pose = PoseStamped()
    pose.header.frame_id = 'map'
    pose.header.stamp = self.get_clock().now().to_msg()
    pose.pose.position.x = waypoint.x
    pose.pose.position.y = waypoint.y
    pose.pose.position.z = 0.0
    pose.pose.orientation.w = 1.0        # identity quaternion — no heading constraint
    poses.append(pose)
msg.poses = poses

self._action_client.wait_for_server()
self._send_goal_future = self._action_client.send_goal_async(msg)
self._send_goal_future.add_done_callback(self._response_callback)
```

The orientation is left as the identity quaternion (`w=1`, all other components implicitly 0) since the survey only cares about the robot's *position* at each RSSI sample point, not its heading — Nav2's planner is free to approach each waypoint from whatever direction is most efficient.

7.3 Action lifecycle

The client is fully asynchronous and event-driven, chaining three callbacks:

- `_send_next_batch()` sends a goal and registers `_response_callback`.
- `_response_callback()` fires once the server accepts the goal; it requests the eventual result and registers `_result_callback`.
- `_result_callback()` fires once Nav2 has physically visited every pose in the batch; it increments `batch_index` and immediately calls `_send_next_batch()` again.

This recursive callback chain continues until `_send_next_batch()` finds no waypoints left to send, at which point it publishes an `Empty` message on `/heatmap_generator_trigger` (the signal the heatmap generator is waiting for to begin interpolation — see §11), tears down the waypoint-visualization subprocess, and shuts the node down.

8. RSSI Simulation Model C++

This is the physics core of the project, implemented as a Nav2 **WaypointTaskExecutor plugin** — a pluginlib-exported class that Nav2's `waypoint_follower` node calls automatically every time it arrives at a waypoint during a `FollowWaypoints` goal, with no direct coupling to the waypoint-generation code at all.

8.1 Why a plugin, and how it's wired in

```
class ReadRssiAtWaypoint : public nav2_core::WaypointTaskExecutor
{
public:
  void initialize(const rclcpp_lifecycle::LifecycleNode::WeakPtr & parent,
                  const std::string & plugin_name) override;
  bool processAtWaypoint(const geometry_msgs::msg::PoseStamped & curr_pose,
                         const int & curr_pose_index) override;
  ...
};

PLUGINLIB_EXPORT_CLASS(
  nav2_read_rssi_at_waypoint::ReadRssiAtWaypoint,
  nav2_core::WaypointTaskExecutor)
```

Registering the class via `PLUGINLIB_EXPORT_CLASS` and declaring `waypoint_task_executor_plugin: "nav2_read_rssi_at_waypoint"` in `nav2_params.yaml` lets the stock, unmodified Nav2 `waypoint_follower` executable load and invoke this class dynamically at runtime — no Nav2 source code is patched. `processAtWaypoint()` is Nav2's per-waypoint callback hook; everything the RSSI model does happens inside it.

8.2 Log-distance path-loss model

File: `nav2_read_rssi_at_waypoint_plugin/include/nav2_read_rssi_at_waypoint/simulated_rssi.hpp`. WiFi signal attenuation with distance is modelled using the standard **log-distance path-loss** formula from wireless communications theory:

```
RSSI(d) = tx_power − 10·n·log10(d / d0)
```

where `d` is the Euclidean distance between the robot and the access point, `d0` is a 1 metre reference distance (distance is floor-clamped to `d0` to avoid a divergent `log10(0)` at very close range), `tx_power` is the reference transmit power in dBm at that 1 metre distance, and `n` is the path-loss exponent — a physically meaningful tunable that models the environment (≈2 for free-space propagation, 3–4 for a cluttered indoor environment with walls and furniture).

8.3 Stochastic noise and range clamping

Real WiFi measurements fluctuate due to multipath fading, interference, and receiver noise — a purely deterministic model would produce unrealistically "clean" heatmaps. A Gaussian noise term is added using the C++ `<random>` library's `std::mt19937` Mersenne-Twister engine and `std::normal_distribution` (μ=0, σ=2 dBm), then the result is rounded to the nearest integer dBm and hard-clamped to the physically valid range for RSSI, `[−100, 0]` dBm:

```
double rssi = tx_power - 10.0 * n * std::log10(dist / d0);

static std::random_device rd;
static std::mt19937 gen(rd());
std::normal_distribution<double> noise(0.0, 2.0);
rssi += noise(gen);

int rssi_rounded = static_cast<int>(std::lround(rssi));
if (rssi_rounded > 0) rssi_rounded = 0;
else if (rssi_rounded < -100) rssi_rounded = -100;
return rssi_rounded;
```

The random engine is declared `static` so it is seeded once (via `std::random_device`) and reused across calls rather than re-seeding on every sample, which would otherwise bias the noise distribution and be far more expensive per call.

8.4 Multi-sample averaging at each waypoint

File: `nav2_read_rssi_at_waypoint_plugin/src/read_rssi_at_waypoint.cpp`, inside `processAtWaypoint()`. A single noisy sample is not representative of the "true" signal at a location, so the plugin takes `number_of_measurements` independent samples (a configurable parameter, default 5), spaced 50 ms apart via `std::this_thread::sleep_for` to emulate the cadence of a real radio doing repeated measurements, discards any sample that comes back invalid (a positive RSSI value, which is not physically meaningful and only clamps at 0 by construction — this is defensive filtering for the boundary case), and reports the arithmetic mean of the remaining valid samples. If every sample in a batch were somehow invalid, the code falls back to reporting −100 dBm (worst-case / no-signal) rather than an undefined average.

9. Bonus 1 — Multi-Access-Point Support Bonus

The base specification models a single WiFi access point. Real buildings have many, with overlapping coverage — a waypoint near the edge of one AP's range might be well inside another's. This bonus generalizes the entire RSSI pipeline from one AP to an arbitrary configured list of APs.

9.1 Parameter shape change: scalar → array

Every physical parameter that used to be a single `double` (`ap_x`, `ap_y`, `tx_power`, `path_loss_exponent`) became a `std::vector<double>`, with index `i` across all four vectors describing access point `i`:

```
protected:
  std::vector<double> aps_x_;
  std::vector<double> aps_y_;
  std::vector<double> tx_powers_;
  std::vector<double> path_loss_exponents_;
```

A small helper, `param_at(values, index)`, lets `tx_power` / `path_loss_exponent` be specified with fewer entries than there are APs (falling back to the last provided value) — so a deployment with uniform transmit power across APs only needs to list AP positions, not repeat the same power value N times. AP position lists (`ap_x` / `ap_y`) are required to match in length; a mismatch throws at plugin initialization rather than silently misbehaving at runtime.

9.2 Strongest-signal selection per waypoint

At each waypoint, `processAtWaypoint()` now loops over *every* configured access point, runs the full multi-sample-averaging procedure from §8.4 independently for each one (using that AP's own position, transmit power, and path-loss exponent), and keeps track of whichever AP produced the strongest (numerically highest / closest-to-zero) averaged RSSI:

```
int best_rssi = -101;
size_t best_ap = 0;

for (size_t ap = 0; ap < aps_x_.size(); ++ap) {
  double tx_power = param_at(tx_powers_, ap);
  double path_loss_exponent = param_at(path_loss_exponents_, ap);
  // ... n_measurements_ samples via simulate_rssi(), averaging valid ones ...
  if (avg_rssi > best_rssi) {
    best_rssi = avg_rssi;
    best_ap = ap;
  }
}

msg.rssi = best_rssi;
msg.serving_ap = static_cast<int8_t>(best_ap);
```

This directly implements the "strongest available signal" coverage model requested by the bonus spec: a waypoint between two APs is reported as being served by whichever one actually dominates there, which is what a real WiFi client does when roaming between access points.

9.3 Interface change & reporting

A new field, `int8 serving_ap`, was added to `rosbot_interfaces/msg/RssiAtWaypoint.msg` to carry the winning AP's index downstream. The heatmap generator subscribes to this alongside `rssi`, accumulates a per-AP waypoint count, and logs an aggregate coverage-by-AP breakdown (plus mean RSSI across the whole survey) at the end of the run — directly satisfying the bonus requirement for "basic statistics such as the access point serving each waypoint."

```
ap_counts = {}
for ap in self.serving_aps:
    ap_counts[ap] = ap_counts.get(ap, 0) + 1
self.get_logger().info(
    f'Survey stats — {len(self.rssi_data)} waypoints, '
    f'avg RSSI={sum(rssi_values)/len(rssi_values):.1f} dBm, '
    f'coverage by AP: {ap_counts}')
```

9.4 Configuration example (two APs)

```
nav2_read_rssi_at_waypoint:
  plugin: "nav2_read_rssi_at_waypoint::ReadRssiAtWaypoint"
  enabled: True
  number_of_measurements: 5
  ap_x: [0.0, 3.0]
  ap_y: [0.0, -2.0]
  tx_power: [-30.0, -30.0]
  path_loss_exponent: [3.0, 3.0]
```

10. Bonus 2 — Nearest-First Dynamic Routing Bonus

The base implementation sends waypoints in raster-scan order (the order they were discovered while striding across the image row by row) — which frequently produces long, zig-zagging travel paths. This bonus replaces that static ordering with dynamic, greedy nearest-neighbour route planning driven by the robot's actual live position.

10.1 Getting a live robot pose via TF2

File: `waypoint_publisher_package/waypoint_publisher_package/node.py`. The node attaches a `tf2_ros.Buffer` and `TransformListener` at construction time, and looks up the `map→base_link` transform (populated at runtime by Nav2's AMCL localization + the robot's URDF/TF tree) on demand:

```
self.tf_buffer = tf2_ros.Buffer()
self.tf_listener = tf2_ros.TransformListener(self.tf_buffer, self)

def _get_robot_position(self):
    try:
        transform = self.tf_buffer.lookup_transform('map', 'base_link', Time())
        return (transform.transform.translation.x, transform.transform.translation.y)
    except TransformException as ex:
        self.get_logger().warn(f'Could not look up robot pose: {ex}')
        return None
```

If the transform is not yet available (e.g. the robot hasn't been localized yet when the very first batch is requested), the code falls back to the map's declared origin rather than crashing, trading a slightly sub-optimal first batch ordering for robustness.

10.2 Greedy nearest-neighbour batch construction

Before each batch is sent, `_pop_nearest_batch()` repeatedly selects whichever *remaining, unvisited* waypoint is closest (by squared Euclidean distance — no square root needed since only the ordering matters) to the current position, removes it from the pool, and updates "current position" to that waypoint before searching again. This is the classic greedy nearest-neighbour heuristic for the (NP-hard) travelling-salesman problem: not globally optimal, but computed in effectively linear time per step and dramatically better than an arbitrary/raster order in practice.

```
def _pop_nearest_batch(self, current_pos):
    batch = []
    position = current_pos
    while self.remaining_waypoints and len(batch) < self.batch_size:
        nearest = min(
            self.remaining_waypoints,
            key=lambda wp: (wp.x - position[0]) ** 2 + (wp.y - position[1]) ** 2)
        self.remaining_waypoints.remove(nearest)
        batch.append(nearest)
        position = (nearest.x, nearest.y)
    return batch
```

Because this re-queries the live TF pose at the start of every batch (not just once at start-up), the ordering genuinely adapts to where the robot actually ends up after each batch of Nav2 navigation — satisfying the "continuously updates the waypoint list" requirement of the bonus spec — while still batching (rather than one-goal-at-a-time dispatch) to keep the number of action calls and re-planning overhead reasonable.

10.3 Backward-compatible toggle

The behaviour is gated by a new `nearest_first` boolean ROS parameter (default `True`). When disabled, `_send_next_batch()` falls back to the original static-slice-by-`batch_index` logic — so the raster-scan behaviour required by the base (non-bonus) specification is still fully available and was never removed, only extended.

11. Heatmap Interpolation & Visualization Python

File: `rssi_heatmap_generator_package/rssi_heatmap_generator_package/node.py` and its helper module `submodules/generate_heatmap.py`. This is the final stage: turning a sparse set of (pixel_x, pixel_y, rssi) samples collected over the survey into a dense, visually interpretable coverage map.

11.1 Accumulating samples

`rssi_data_callback()` subscribes to `/rssi_data` and, for every incoming `RssiAtWaypoint` message, converts the reported world-frame coordinates into pixel coordinates (§5.1) and appends an `RssiWaypoint(x, y, rssi)` namedtuple to an in-memory list — alongside the reported `serving_ap` for the Bonus 1 statistics described in §9.3. Nothing is interpolated until the survey is actually complete.

11.2 Trigger & radial basis function interpolation

Once the waypoint publisher signals completion on `/heatmap_generator_trigger`, `trigger_callback()` hands the accumulated samples (requiring a minimum of 3, since fewer points cannot support a meaningful spatial interpolation) to `generate_heatmap()`, which uses **SciPy's** **`RBFInterpolator`** — a radial basis function interpolator that fits a smooth continuous surface through scattered, irregularly-spaced data points, exactly the situation here (waypoints are not on a regular grid relative to the full map).

```
RBFdata = np.stack((np.array(xdata).ravel(), np.array(ydata).ravel()), -1)
f = RBFInterpolator(RBFdata, np.array(valdata).ravel())

xpoints, ypoints = np.meshgrid(np.arange(0, x_image_size, 1), np.arange(0, y_image_size, 1))
RBFpoints = np.stack((np.array(xpoints).ravel(), np.array(ypoints).ravel()), -1)
interp_data = f(RBFpoints).reshape(np.array(xpoints).shape)
```

The interpolator is evaluated at *every pixel* of the map (via a full `np.meshgrid`), producing a dense RSSI estimate across the entire environment — including areas the robot never physically visited — from only the discrete waypoint measurements.

11.3 Two normalization modes: absolute vs. relative

`generate_heatmap()` is called twice with different parameters, producing two distinct products:

Heatmap	Colour normalization range	Post-processing	Purpose
Absolute	Fixed physical range, −100 to 0 dBm	None	True signal strength — directly comparable across different surveys/buildings.
Relative	This survey's own observed min/max RSSI	`cv2.medianBlur` (7×7) smoothing	Maximizes contrast within a single survey, making weak-vs-strong areas easier to see even when the whole building has generally poor or generally good coverage.

The relative mode's bounds (`minrssi`, `maxrssi`) are returned alongside the heatmap array specifically so the visualization layer can draw an accurately-scaled colorbar (§11.4) instead of a hard-coded one.

11.4 Colour mapping and map overlay

RSSI values are mapped to colour via a custom red→green `LinearSegmentedColormap` (`cmapGR`, defined in `generate_heatmap.py`) and `matplotlib.cm.ScalarMappable`, giving a familiar "red = bad signal, green = good signal" visual language. The interpolated heatmap is then overlaid onto the *original* occupancy map via `add_heatmap()`, which only replaces pixels that were free space in the source map (intensity 254) — walls and unknown regions are left untouched so the heatmap remains spatially legible against the building's actual layout rather than obscuring it.

11.5 Display and export

`display_maps()` (run in a separate `multiprocessing.Process` so it never blocks the ROS executor) opens three independent `matplotlib` windows — the annotated waypoint QA map, the absolute heatmap, and the relative heatmap — the latter two each built with a `gridspec`-based side panel holding a properly-scaled colorbar via `ScalarMappable`. All three images are additionally written to disk as timestamped PNGs (`waypoints_<timestamp>.png`, `heatmap_abs_<timestamp>.png`, `heatmap_rel_<timestamp>.png`) under the configured `heatmaps_dir`, so successive survey runs never overwrite each other's results.

12. Message & Parameter Reference

12.1 `rosbot_interfaces/msg/RssiAtWaypoint.msg`

```
geometry_msgs/Point coordinates   # world-frame (map) position where the sample was taken
int8 rssi                          # strongest averaged RSSI across all configured APs, dBm [-100, 0]
int8 serving_ap                    # index of the AP that produced that strongest reading
```

12.2 `nav2_read_rssi_at_waypoint` plugin parameters (nav2_params.yaml)

Parameter	Type	Default	Meaning
`enabled`	bool	true	Master on/off switch for RSSI sampling.
`number_of_measurements`	int	5–10	Samples averaged per AP per waypoint (§8.4). Setting to 0 auto-disables the plugin.
`ap_x` / `ap_y`	double[]	`[0.0]`	Map-frame position of each configured access point (§9).
`tx_power`	double[]	`[-30.0]`	Reference transmit power (dBm) at 1 m, per AP.
`path_loss_exponent`	double[]	`[3.0]`	Environment attenuation factor `n` in the path-loss formula (§8.2), per AP.

12.3 `waypoint_publisher_package` node parameters

Parameter	Type	Default	Meaning
`density`	int	8	Grid sampling stride in pixels (§6.1, step 4).
`collision_range`	int	4	Obstacle-mask inflation radius in pixels (§6.1, step 3).
`batch_size`	int	20	Waypoints per `FollowWaypoints` goal (§7.1).
`nearest_first`	bool	true	Enables the Bonus 2 greedy TF-based routing (§10.3).
`path_to_yaml`	string	—	Path to the saved `map.yaml` .

12.4 `rssi_heatmap_generator_package` node parameters

Parameter	Type	Default	Meaning
`path_to_yaml`	string	—	Path to the saved `map.yaml` (must match the publisher's).
`heatmaps_dir`	string	`~/ros2_RSSI_Heatmap/src/heatmaps`	Output directory for exported PNGs (§11.5).

13. Launch Architecture & Run Procedure

The `mappers_bringup` package provides three launch files, corresponding to the three phases of a survey run:

1. `sim_nav2_slam.launch.py` — brings up the Nav2 navigation stack (`controller_server`, `planner_server`, `behavior_server`, `bt_navigator`, `waypoint_follower` with the RSSI plugin already wired in) alongside SLAM Toolbox's `online_async` node and RViz2, so the operator can drive the robot around (teleop) while SLAM builds the map live.
2. After the map is saved via `nav2_map_server`'s `map_saver_cli`, `sim_nav2_localization.launch.py` replaces SLAM with AMCL-based localization against the now-fixed, saved map.
3. `mappers.launch.py` starts the two survey-specific nodes — `waypoint_publisher_package` and `rssi_heatmap_generator_package` — parameterized with the map path, output directory, waypoint density, and safety margin, kicking off the full generate→navigate→sample→interpolate→export sequence autonomously.

```
ros2 launch mappers_bringup sim_nav2_slam.launch.py                # Phase 1: build the map
ros2 run nav2_map_server map_saver_cli -f ~/ros2_RSSI_Heatmap/src/maps/map --ros-args -p use_sim_time:=true
ros2 launch mappers_bringup sim_nav2_localization.launch.py map:=$HOME/ros2_RSSI_Heatmap/src/maps/map.yaml
ros2 launch mappers_bringup mappers.launch.py path_to_yaml:=$HOME/ros2_RSSI_Heatmap/src/maps/map.yaml
```

14. Verification Performed & Known Limitations

> **Honesty note on verification scope.** This machine has ROS 2 Humble installed but **not** the Nav2, TurtleBot3, or SLAM Toolbox packages the project's README instructs users to `apt install` (the README itself targets ROS 2 Jazzy). Installing that full stack requires substantial system-level package changes that were intentionally not performed unprompted. As a result, **no live Gazebo/Nav2 simulation run was executed in this environment**, and the figures in §15 describe expected output rather than a captured run.

What was actually verified:

- Every modified Python file was parsed with `ast.parse()` to confirm it is syntactically valid.
- Every C++ change was manually traced against the actual method signatures in `read_rssi_at_waypoint.hpp`, the message fields generated from `RssiAtWaypoint.msg`, and the Nav2 `WaypointTaskExecutor` interface contract, to confirm type-correctness and control flow by inspection (e.g. confirming `int8_t` fits the plugin's AP index range, and that `param_at()`'s fallback indexing cannot go out of bounds).
- Coordinate-frame math (§5) was independently re-derived from the `map.yaml` convention and cross-checked against both the waypoint publisher's and the heatmap generator's use of it, to confirm the two are exact inverses of each other rather than independently-plausible-but-inconsistent implementations.
- The colour-channel ordering issue described in §6.2 was caught and fixed during implementation precisely because of this manual trace (an initial BGR-style tuple would have rendered rejected waypoints as blue, not red, in the on-screen visualization).

Known limitations / things a full hardware-in-the-loop or simulated run would additionally validate:

- Real Nav2 planner behaviour near the inflated safety margin (whether `collision_range`'s pixel-based margin is generous enough relative to the robot's actual footprint and the costmap's own inflation layer).
- TF availability timing for the Bonus 2 nearest-first lookup during the very first batch, before AMCL has converged from its initial pose estimate.
- RBF interpolation quality/runtime at map resolutions much larger than tested conceptually here — `RBFInterpolator` is $O(N^3)$ in the number of sample points for the fitting step, which is fine for tens-to-low-hundreds of waypoints but would need a compactly-supported kernel or neighbor-limited variant at much higher survey densities.

15. Expected Runtime Output

Based on the implemented logic, a full run of `mappers.launch.py` is expected to produce console output of this shape (values illustrative, since RSSI includes Gaussian noise and depends on the specific map/AP placement used):

```
[waypoint_publisher] 47 valid waypoints generated
[waypoint_publisher] Generated 47 waypoints — sending in batches of 20 (nearest-first)
[waypoint_publisher] Sending batch 1 (20 waypoints, 27 remaining) of 47 total
[waypoint_publisher] Batch 1 accepted by server
[nav2_read_rssi_at_waypoint]   RSSI = -46 dBm at (1.20, 0.85), served by AP 0
[nav2_read_rssi_at_waypoint]   RSSI = -52 dBm at (1.90, 1.10), served by AP 1
...
[waypoint_publisher] Batch 1 complete
[waypoint_publisher] Sending batch 2 (20 waypoints, 7 remaining) of 47 total
...
[waypoint_publisher] All batches complete — publishing trigger...
[heatmap_generator] Survey stats — 47 waypoints, avg RSSI=-58.3 dBm, coverage by AP: {0: 28, 1: 19}
[heatmap_generator] Heatmaps saved successfully.
```

And three files under `~/ros2_RSSI_Heatmap/src/heatmaps/` :

File	Content
`waypoints_<ts>.png`	Occupancy map with every measured waypoint plotted as a colour-mapped RSSI dot (QA/debug view).
`heatmap_abs_<ts>.png`	Full-map RBF-interpolated coverage, colour scale fixed to the physical −100…0 dBm range.
`heatmap_rel_<ts>.png`	Full-map RBF-interpolated coverage, colour scale stretched to this survey's own observed min/max, median-blurred.

16. Summary of Changes (File-by-File)

File	Nature of change
`waypoint_publisher_package/waypoint_publisher_package/node.py`	Implemented waypoint generation (obstacle mask + inflation + grid sampling), Nav2 goal construction (§7), and the full Bonus 2 TF-based nearest-neighbour routing subsystem (§10).
`waypoint_publisher_package/package.xml`	Added the `tf2_ros` dependency required by Bonus 2.
`nav2_read_rssi_at_waypoint_plugin/include/.../simulated_rssi.hpp`	Implemented the log-distance path-loss + Gaussian noise + clamping RSSI model (§8.2–8.3).
`nav2_read_rssi_at_waypoint_plugin/src/read_rssi_at_waypoint.cpp`	Implemented per-waypoint multi-sample averaging (§8.4) and generalized it to loop over and select the strongest of multiple configured APs (§9.2).
`nav2_read_rssi_at_waypoint_plugin/include/.../read_rssi_at_waypoint.hpp`	Changed AP parameter members from scalars to `std::vector<double>` (§9.1).
`rosbot_interfaces/msg/RssiAtWaypoint.msg`	Added the `serving_ap` field (§9.3).
`rssi_heatmap_generator_package/rssi_heatmap_generator_package/node.py`	Implemented pixel-coordinate conversion (§5.1), the previously-stubbed `display_maps()` three-window visualization with colorbars (§11.5), and the per-AP survey statistics logging (§9.3).
`mappers_bringup/config/nav2_params.yaml`	Reconfigured the RSSI plugin's parameters from single-AP scalars to a two-AP array example (§9.4).

End of report. This document was generated to accompany the completed `ros2_RSSI_Heatmap` implementation, including both optional bonus challenges from the project specification.